

Day 1 - Complete Revision Notes

Arrays + Big-O + Hashing Foundations | Java-focused | Senior backend interview bridge

Day 1 target: Recognize the first pattern family: Array -> HashSet -> HashMap. Be able to explain not only what code works, but why it works, its complexity, and what changes when the problem moves into a real backend system.

1. Day 1 at a glance

Block	What to learn	Core outcome
DSA	Concatenation, Running Sum, Contains Duplicate, Valid Anagram, Two Sum	Array traversal + lookup patterns
Big-O	$O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$, $O(n!)$	Read complexity quickly
Java	<code>int[]</code> , <code>Integer[]</code> , Arrays utilities, array copying/search	Use Java arrays confidently
Hashing	<code>hashCode</code> , bucket, collision, equals, resize, treeification	Understand HashMap internals
Backend	List vs HashMap vs DB index vs Redis	Connect DSA to architecture

Core rule: do not memorize “HashMap = $O(1)$ ”. Learn the mechanism: key -> hash -> bucket -> collision handling -> equals -> value.

2. DSA - Arrays and the first pattern family

2.1 Array fundamentals

An array stores elements in indexed positions. For an array reference `arr`, accessing `arr[i]` is constant-time because the location is computed from the base address and index (the exact memory layout is JVM implementation-specific). Searching an unsorted array is $O(n)$ because the target may be anywhere or absent.

Operation	Typical complexity	Why
<code>arr[i]</code>	$O(1)$	Direct indexed access
scan all elements	$O(n)$	May inspect every element
search unsorted array	$O(n)$	No ordering information
binary search on sorted array	$O(\log n)$	Discard about half each step
sort	Often $O(n \log n)$	Depends on algorithm / Java type
copy n elements	$O(n)$	Each element must be copied

2.2 NeetCode: Concatenation of Array

Goal: Given `nums` of length n , return `nums + nums`. The output has $2n$ elements. This problem is simple, but it trains clean array traversal and complexity reasoning.

```
class Solution {
    public int[] getConcatenation(int[] nums) {
        int n = nums.length;
        int[] ans = new int[2 * n];

        for (int i = 0; i < n; i++) {
            ans[i] = nums[i];
            ans[i + n] = nums[i];
        }

        return ans;
    }
}
```

Brute-force thought: there is no asymptotically better approach because the result itself contains $2n$ values. A two-loop solution is still $O(n)$; one loop is simply cleaner.

Complexity: Time $O(n)$, output/extra space $O(n)$.

Edge cases: one element, repeated values, negatives; the original problem constraint excludes empty input.

Verbal answer: “I allocate an output array of size $2n$ and place `nums[i]` at i and $i+n$. The work is linear, and linear time is optimal because I must produce $2n$ output elements.”

2.3 Running Sum - one-pass state

Input: `[1, 2, 3, 4]`

Output: [1, 3, 6, 10]

```
// In-place version
for (int i = 1; i < nums.length; i++) {
    nums[i] += nums[i - 1];
}
```

Pattern: scan once + maintain state. Time $O(n)$. In-place version uses $O(1)$ auxiliary space (it reuses the input array).

2.4 Contains Duplicate - introduce HashSet

```
Set<Integer> seen = new HashSet<>();
```

```
for (int num : nums) {
    if (seen.contains(num)) {
        return true;
    }
    seen.add(num);
}
return false;
```

Pattern: if the question is “Have I seen this before?”, think HashSet. Brute force is $O(n^2)$; HashSet gives expected $O(n)$ overall with $O(n)$ space.

2.5 Valid Anagram - introduce frequency counting

```
if (s.length() != t.length()) return false;
Map<Character, Integer> count = new HashMap<>();

for (char c : s.toCharArray()) {
    count.put(c, count.getOrDefault(c, 0) + 1);
}
for (char c : t.toCharArray()) {
    count.put(c, count.getOrDefault(c, 0) - 1);
}
```

Pattern: frequency map. Sorting both strings gives $O(n \log n)$; counting gives expected $O(n)$. If the alphabet is fixed (for example 26 lowercase letters), an `int[26]` can make auxiliary space $O(1)$.

2.6 Two Sum - turn search into lookup

```
Map<Integer, Integer> seen = new HashMap<>();
```

```
for (int i = 0; i < nums.length; i++) {
    int need = target - nums[i];

    if (seen.containsKey(need)) {
        return new int[] { seen.get(need), i };
    }

    seen.put(nums[i], i);
}
return new int[0];
```

Key idea: Instead of searching the remaining array for the complement, remember what has already appeared. For each current value x , the needed value is $\text{target} - x$. This changes expected time from $O(n^2)$ to $O(n)$ at the cost of $O(n)$ extra space.

3. Big-O - read code by growth rate

3.1 The seven complexities

Complexity	Typical pattern	Mental model
$O(1)$	<code>arr[i]</code> , fixed work	Does not grow with n
$O(\log n)$	binary search, $i *= 2$	Problem shrinks by a factor
$O(n)$	single scan	One pass through input
$O(n \log n)$	efficient comparison sort	$\log n$ levels $\times n$ work
$O(n^2)$	all pairs / nested scans	$n \times n$
$O(2^n)$	all subsets / two choices per item	Branch twice per item
$O(n!)$	all permutations	n choices, then $n-1$...

3.2 The five rules

- Sequential blocks add: $O(n) + O(n) = O(2n) = O(n)$.
- Nested loops multiply: $O(n) \times O(n) = O(n^2)$.
- Drop constants: $O(5n) = O(n)$.

- Keep the dominant term: $O(n^2 + n + 100) = O(n^2)$.
- Use separate variables when inputs differ: an n -loop inside an m -loop is $O(nm)$, not $O(n^2)$.

3.3 Interview drills

```
// A: triangular nested loop
for (int i = 0; i < n; i++) {
    for (int j = i; j < n; j++) { }
}
Answer:  $O(n^2)$ 
Reason:  $n + (n-1) + \dots + 1 = n(n+1)/2$ 
```

```
// B: doubles each iteration
for (int i = 1; i < n; i *= 2) { }
Answer:  $O(\log n)$ 
```

```
// C: linear outer loop + logarithmic inner loop
for (int i = 0; i < n; i++) {
    for (int j = 1; j < n; j *= 2) { }
}
Answer:  $O(n \log n)$ 
```

3.4 Fast-recognition patterns

One loop: usually $O(n)$. Loop variable doubles/halves: usually $O(\log n)$. Every pair: usually $O(n^2)$. Sort then scan: usually $O(n \log n)$. Every subset: $O(2^n)$. Every permutation: $O(n!)$.

3.5 Complexity is not exact runtime

Big-O describes asymptotic growth. It does not mean an $O(1)$ operation is always faster in wall-clock time than an $O(n)$ operation for small n . Real latency also depends on constants, cache locality, allocations, network trips, disk I/O, CPU, contention, and implementation details.

4. Java Arrays - what you must know

4.1 int[] vs Integer[]

Aspect	int[]	Integer[]
Element type	primitive int	reference to Integer object
Default element	0	null
Object overhead	lower	higher in general
Boxing/unboxing	no	may occur
Collections compatibility	not directly	works with List/Map/Set generics
Large numeric data	usually more memory-efficient	more overhead

Important: arrays themselves are objects. For reference arrays, the array stores references; the referenced objects have their own object representation. Exact memory layout is JVM-specific.

4.2 Core Java APIs

```
Arrays.sort(nums);
Arrays.copyOf(nums, newLength);
Arrays.fill(nums, -1);
Arrays.binarySearch(sortedNums, key);
System.arraycopy(src, srcPos, dst, dstPos, length);
Arrays.toString(nums);
Arrays.deepToString(matrix);
```

API	Remember
Arrays.sort	Sorts array; modifies array
Arrays.copyOf	Creates a new array; truncates/pads
Arrays.fill	Fills a range/all elements with one value
Arrays.binarySearch	Use on appropriately sorted data
System.arraycopy	Copies a range from one array to another

4.3 Common Java interview traps

- `System.out.println(nums)` does not give a useful element list; use `Arrays.toString(nums)`.
- `binarySearch` on unsorted input is not a valid general search strategy.
- List vs int[]: collections use objects, so boxing/object overhead can matter.

- Output space vs auxiliary space: an output array required by the problem may be unavoidable; distinguish it from extra working memory.

5. Hashing - the missing foundation

5.1 Why hashing exists

A list forces you to search. A hash table tries to turn a key into a location. Instead of “scan everything”, the mental model becomes “compute where the key should live”.

Pipeline

key -> hashCode() -> internal hash/spread -> bucket index -> bucket -> collision handling -> equals() -> value

5.2 Hash code vs bucket index

In Java, hashCode() returns an int hash code. That integer is not necessarily the final bucket number. HashMap further mixes/spreads the bits and derives an index into its internal table. Current OpenJDK uses power-of-two table sizes and an index calculation based on bit masking.

5.3 Collision

Example teaching hash function:

$h(\text{key}) = \text{key} \% 10$

$12 \% 10 = 2$

$22 \% 10 = 2$

Both keys -> bucket 2

That is a collision.

A collision is normal. Multiple different keys can map to the same bucket. A good hash distribution reduces how often and how badly this happens.

5.4 Collision handling in HashMap

Bucket 2

```
|
+--> (12, Alice)
|
+--> (22, Bob)
|
+--> (32, Charlie)
```

Lookup 22:

key -> hash -> bucket 2 -> candidates -> compare -> value

Historically, collision bins are described as separate chaining. Modern OpenJDK uses nodes in bins and can treeify a heavily-colliding bin. The exact implementation is a Java implementation detail, not the API contract.

5.5 hashCode() and equals() - the golden contract

If `a.equals(b)` is true, then `a.hashCode()` MUST equal `b.hashCode()`. The reverse is not required: same hash does not imply equality.

```
// Modern Java pattern
@Override
public boolean equals(Object obj) {
    if (!(obj instanceof BadKey other)) {
        return false;
    }
    return value == other.value;
}

// Why other.value?
// obj is declared as Object, so obj.value does not compile.
// instanceof both checks the type and binds a BadKey reference named `other`.
// `value` means this.value.
```

Equivalent older syntax: `BadKey other = (BadKey) obj;` after the type check, then compare `value == other.value`.

5.6 Why hashCode() alone is not enough

Alice -> hash 42
Bob -> hash 42

Same hash, different objects/keys.

Hash narrows the candidate bucket; equals() identifies the exact key.

5.7 Mutable-key trap

Do not mutate fields used by equals()/hashCode() while the object is a HashMap key. Otherwise the object can remain in its old bucket while future lookups calculate a new bucket, making the mapping effectively hard to find.

5.8 Complexity - the correct interview wording

Best answer: "HashMap get/put/remove are expected $O(1)$ with a good hash distribution. Collisions can make lookup slower. Modern Java can treeify heavily-colliding bins, but the exact internal behavior is implementation-specific."

5.9 Load factor and resizing

Java HashMap uses a load factor to decide when the table should grow. The default load factor is 0.75 in current Java. Roughly: when entries exceed capacity $\times$ load factor, resizing is triggered. Resizing increases the table capacity and redistributes entries. This trades memory for shorter collision chains.

5.10 Treeification

Current OpenJDK HashMap uses tree bins for sufficiently large collision groups. The source defines a treeification threshold of 8 and a minimum table capacity of 64 before treeification is preferred. Treat these as implementation details that can vary by Java release, not as the core HashMap contract.

6. HashSet vs HashMap

Need	Use	Mental question
Existence / uniqueness	HashSet	"Have I seen this?"
Key -> value	HashMap	"What value belongs to this key?"
Frequency	HashMap or int[]	"How many times?"
Remember index by value	HashMap	"Where did I see this?"

6.1 The pattern family

Array

Need to inspect data? Traverse it.

HashSet

Need fast membership/duplicate detection?

HashMap

Need key -> value, frequency, complement, or stored index?

6.2 Two Sum as the canonical HashMap pattern

```
nums = [2, 7, 11, 15], target = 9
```

```
current = 2
```

```
need = 9 - 2 = 7
```

```
Have we seen 7? No -> remember 2
```

```
current = 7
```

```
need = 9 - 7 = 2
```

```
Have we seen 2? Yes -> answer found
```

This is the broader pattern: replace repeated search with remembered information. That is the time-space tradeoff you will use repeatedly in NeetCode problems.

7. Senior backend exercise - finding a user among 10 million

Option	Lookup	Persistence	Shared/distributed	Typical role
List scan	$O(n)$	No	No	Small local collection
HashMap	$O(1)$ expected	No	No	Fast process-local lookup/cache
DB + index	Typically $O(\log n)$ for B-tree lookup	Yes	Yes	System of record
Redis	$O(1)$ for common key operations	Configurable	Yes	Shared cache / fast data layer

Do not choose by Big-O alone. A local HashMap avoids a network hop; Redis is shared across application instances; a database provides durability, transactions, constraints and indexing. Real latency also includes network, serialization, cache hit/miss behavior, database planning, I/O and contention.

7.1 Decision mental model

HashMap: "I need very fast local lookup."

Redis: "I need a shared fast data layer/cache."

Database: "This data is durable and authoritative."

List: "The collection is small, or I truly need ordered/sequential traversal."

7.2 Cache-aside flow



8. Interview drill - answer without code

Question	What a strong answer should contain
Why is <code>arr[i]</code> $O(1)$?	Indexed access does not require scanning the array; location is derived from index.
Why is Two Sum $O(n)$ with <code>HashMap</code> ?	One pass + constant-time expected lookup per element; extra $O(n)$ memory.
Why is <code>HashMap</code> expected $O(1)$?	Hash narrows to a bucket; good distribution keeps candidate count small.
What is a collision?	Different keys map to the same bucket; collision handling stores/organizes multiple entries.
Why both <code>hashCode()</code> and <code>equals()</code> ?	<code>hashCode</code> locates the candidate bucket; <code>equals</code> identifies the exact logical key.
Can equal objects have different hashes?	No. Equal objects must have the same hash code.
Can equal hashes mean different objects?	Yes. Hash collisions are allowed.
Why resize <code>HashMap</code> ?	To reduce crowding/collisions as the table grows, trading memory and rehash work for lookup performance.
Why <code>int[]</code> instead of <code>Integer[]</code> ?	Primitive storage avoids per-element object/reference/boxing overhead.
Why Redis vs <code>HashMap</code> ?	Redis is shared across instances and survives process boundaries; <code>HashMap</code> is local JVM memory.
Why DB index vs <code>HashMap</code> ?	DB index is persistent and queryable by the database; <code>HashMap</code> is local and non-durable.

9. Day 1 checklist

- Code Concatenation of Array from a blank editor.
- Code Running Sum, Contains Duplicate, Valid Anagram and Two Sum from memory.
- Classify $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$, $O(n!)$.
- Explain triangular nested loops as $O(n^2)$, doubling loops as $O(\log n)$, and nested $n \times \log n$ loops as $O(n \log n)$.
- Explain `int[]` vs `Integer[]`.
- Explain hash -> bucket -> collision -> equals -> value without notes.
- State the equals/hashCode contract correctly.
- Explain load factor, resize and the purpose of treeification at a high level.
- Explain List vs `HashMap` vs DB index vs Redis using performance + persistence + distribution + concurrency.

10. External references

Resource	Link
NeetCode - Concatenation of Array	Open problem
Oracle - <code>HashMap</code> API	Open API
OpenJDK - <code>HashMap</code> source	Open source
Oracle - Arrays API	Open API

Resource	Link
VisuAlgo - Hash Table	Open visualizer
PostgreSQL - Index Types	Open docs
Redis - Data Types	Open docs

One-minute memory aid: Array = indexed storage. HashSet = membership. HashMap = key -> value. Hashing = key -> bucket. Collision = multiple entries in one bucket. hashCode narrows; equals confirms. Load factor controls growth. Big-O tells growth; architecture adds persistence, network, concurrency and operational trade-offs.